BOWIE STATE UNIVERSITY

COSC 369 | Software Engineering I

Faculty Teaching and Office-Hours Scheduler

Project report / working C implementation

Source-based project demonstration: Fall 2026 CTEC workbook; COSC 369 syllabus; COSC 369 project assignment.

The included program and outputs are a reproducible demonstration. Class schedules are extracted from provided sources; faculty office hours are reported only where explicitly provided.

1. Problem and requirements

The assignment requires a modular program in C that reads exactly two files: (1) a course/class schedule and (2) faculty information. The program must generate a professor teaching schedule and office-hours schedule, compile reusable code into a static library, and use a Makefile for automated compilation and output generation. A README, generated samples, code, input files, and a PDF explanation are also required.

The COSC 369 syllabus frames the project around software design, data input, analysis, reporting, and documenting a working system. The assignment additionally demonstrates optional Python extraction and Prolog knowledge representation.

2. Input sources and provenance

Runtime file	Source and interpretation
data/courses.csv	56 CTEC spreadsheet rows, plus one distinct COSC 369 row from syllabus / project sample.
data/faculty.csv	28 CTEC instructor names with office hours unknown, plus Dr. Appolo Tankeh with explicitly stated office hours.

The CTEC workbook identifies its instructor column as “Instructor 2025” despite the Fall 2026 filename. Those course/instructor associations are therefore source observations, not verified Fall 2026

faculty assignments. The COSC 369 syllabus has a Spring 2026 heading and Fall 2026 weekly dates; this conflict was not silently corrected.

The two files are stored locally for offline reproducibility. C uses fopen/fgets to read both. The workbook was converted once into CSV to keep the C runtime independent of Excel packages. Python utilities run only after the C program produces output.

3. Static library, modules, and Makefile

A static library is an archive of compiled object files linked into an executable at build time. Here src/scheduler.c is compiled to build/scheduler.o; "ar rcs libschedule.a build/scheduler.o" creates the archive. The main module links with "gcc ... -L. -lschedule -o scheduler". The Makefile records file dependencies and recompiles changed source files. "make test" builds, executes, and verifies the output; "make report" additionally generates Prolog facts and a readable schedule.

The modular design keeps command-line control in main.c, public declarations in scheduler.h, reusable CSV parsing/validation/report generation in scheduler.c, optional Prolog export in generate_facts.py, and verification in tests/test_outputs.py.

4. Scheduling and conflict rules

The scheduler writes every course record with instructor, days, start/end time, room, source, and a validation status. For each faculty office-hours record with specified times, it finds classes for the same exact instructor name. If an office-hours interval overlaps any class on a shared weekday, the slot is marked CONFLICT; missing office hours remain "unknown - not supplied". The program does not infer working hours or invent faculty availability.

Meetings without start/end times are labeled asynchronous / no fixed meeting; malformed or non-increasing clock times are marked for manual review, not repaired by guessing. These checks respect the assignment's statement that instructors should not be interrupted during classes.

5. Sample answers and output

Question	Supported answer
Who teaches COSC 369?	Dr. Appolo Tankeh

When is COSC 369?	Tuesday / Thursday, 9:30 AM–10:45 AM
Where is COSC 369?	CS 108 (project sample, also syllabus office location)
What are COSC 369 office hours?	Tuesday / Thursday, 10:45 AM–1:00 PM

The sample project assignment also illustrates COSC 631 and COSC 330 classes and Monday/Wednesday office hours. These are display examples only, so this implementation does not hard-code them as instructor-confirmed data.

6. Python and Prolog roles

The C executable is the required scheduling application. As optional postprocessing, Python reads C-generated CSV output and creates Prolog clauses. PySwip provides access to a separately installed SWI-Prolog engine. A query such as who_teaches('COSC 369',Professor) retrieves the instructor from generated facts. The C scheduler runs independently of Python, PySwip, or SWI-Prolog.

7. Testing results and limits

The student executed the project from a WSL terminal in VS Code. The screenshot of make test records: "Loaded 57 courses and 29 faculty records"; "Provided office-hour records: 1; teaching overlaps: 0; nonstandard course-time records: 5"; and a PASS for course rows, COSC 369 facts, office hours, asynchronous records, and anomalous times. These are observed program outputs, not a claim that all possible scheduling cases have been verified. In a separate execution the Python-PySwip query reported Dr. Appolo Tankeh as COSC 369 instructor and returned the TR 10:45-13:00 office-hour record. SWI-Prolog issued discontiguous-predicate warnings; these did not stop the query.

8. Deliverables and instructions

The separate project ZIP contains the C source files, public header, Makefile, static library, Linux executable, the two runtime input CSV files, Python scripts, generated CSV schedules, Prolog knowledge base, tests, and README. The executable is built on Linux/WSL with make; make test runs validation; make report regenerates C schedule outputs and Python-derived artifacts. For final submission, attach this report as PDF alongside the full ZIP or as instructed by the lecturer.

10. Requirement-by-requirement coverage

The assignment is written mainly as project requirements rather than a numbered theory examination. This checklist cross-references each explicit requirement to its answer or delivered artifact. The three documents provide source data, while definitions of C build tools and design decisions are technical explanations supplied for the assignment.

Assignment requirement	How addressed
Design a working scheduling system in C	src/main.c, src/scheduler.c and compiled scheduler; generates CSV schedules.
Read exactly two runtime input files	data/courses.csv and data/faculty.csv are the only C command-line data inputs.
Teaching schedules and office hours	output/teaching_schedule.csv and output/office_hours.csv; unknown hours are explicitly marked.
Use separate modules and header files	src/main.c, src/scheduler.c and include/scheduler.h separate entry point, operations and interface.
Explain and create a static library	Section 3; libschedule.a contains scheduler.o; archive listing verified by ar t.
Explain static linking	Makefile links main.o with -L. -lschedule; library code is included when executable is linked.
Makefile and dependency automation	Makefile targets all, run, facts, report, test, clean with compile/archive/link commands.
Local versus Internet input decision	Section 2: local CSV files are selected for offline, reproducible access.
Explain file access	C opens both user-specified CSV paths, parses data and writes generated output.
Python if used	scripts/generate_facts.py derives Prolog facts; export_report.py generates text summary.

Prolog role and sample questions	Section 6 and evidence: teaches(COSC369) and office_hour facts queried using PySwip.
Sample tables not hardcoded	Course records originate in workbook/syllabus; sample COSC 330/631 rows are not inserted as actual data.
README and sample output	README.md and output directory included in accompanying ZIP.
PDF report with explanations	Editable Word report can be exported to PDF after final review.

11. Build, execution and test evidence

Screenshots below were supplied from the WSL/VS Code execution session. They document observed operation, not exhaustive formal verification. The VS Code warning that a file cannot be displayed as text is caused by opening the compiled binary scheduler, not by a scheduling failure.

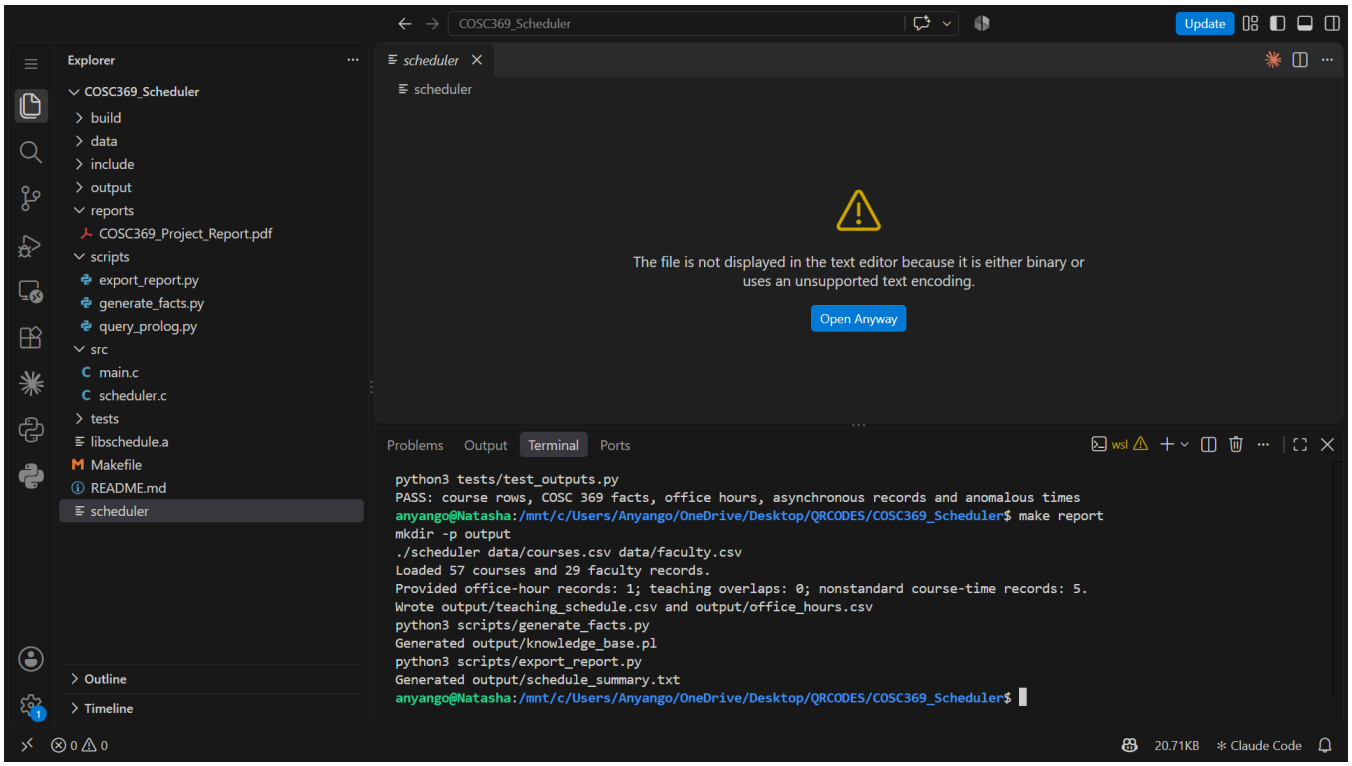


Figure 1. Successful make test. The automated checks passed, with 57 course records, 29 faculty records, one provided office-hours record, no detected teaching overlaps and five flagged nonstandard time records.

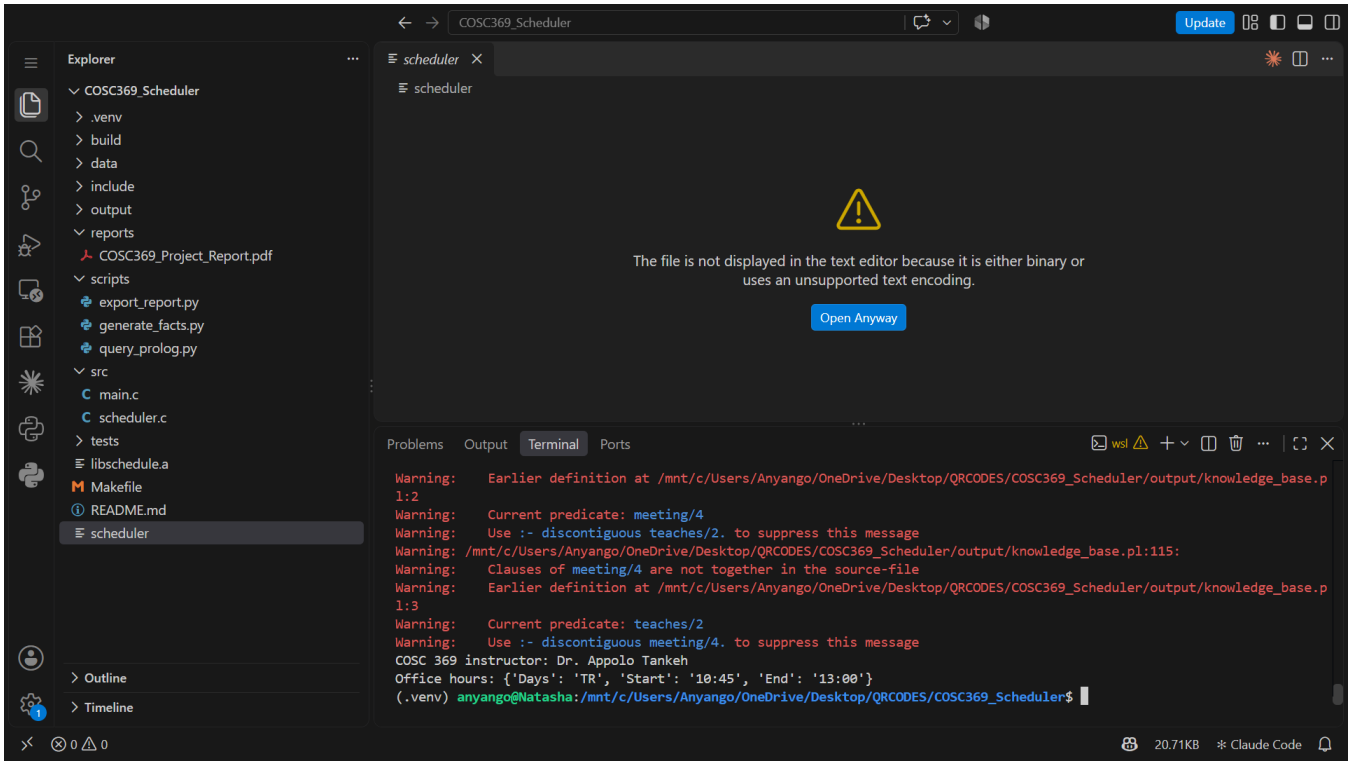


Figure 2. PySwip query result. The script returned the COSC 369 instructor and Dr. Tankeh office hours; Prolog discontiguous-clause warnings remain visible.

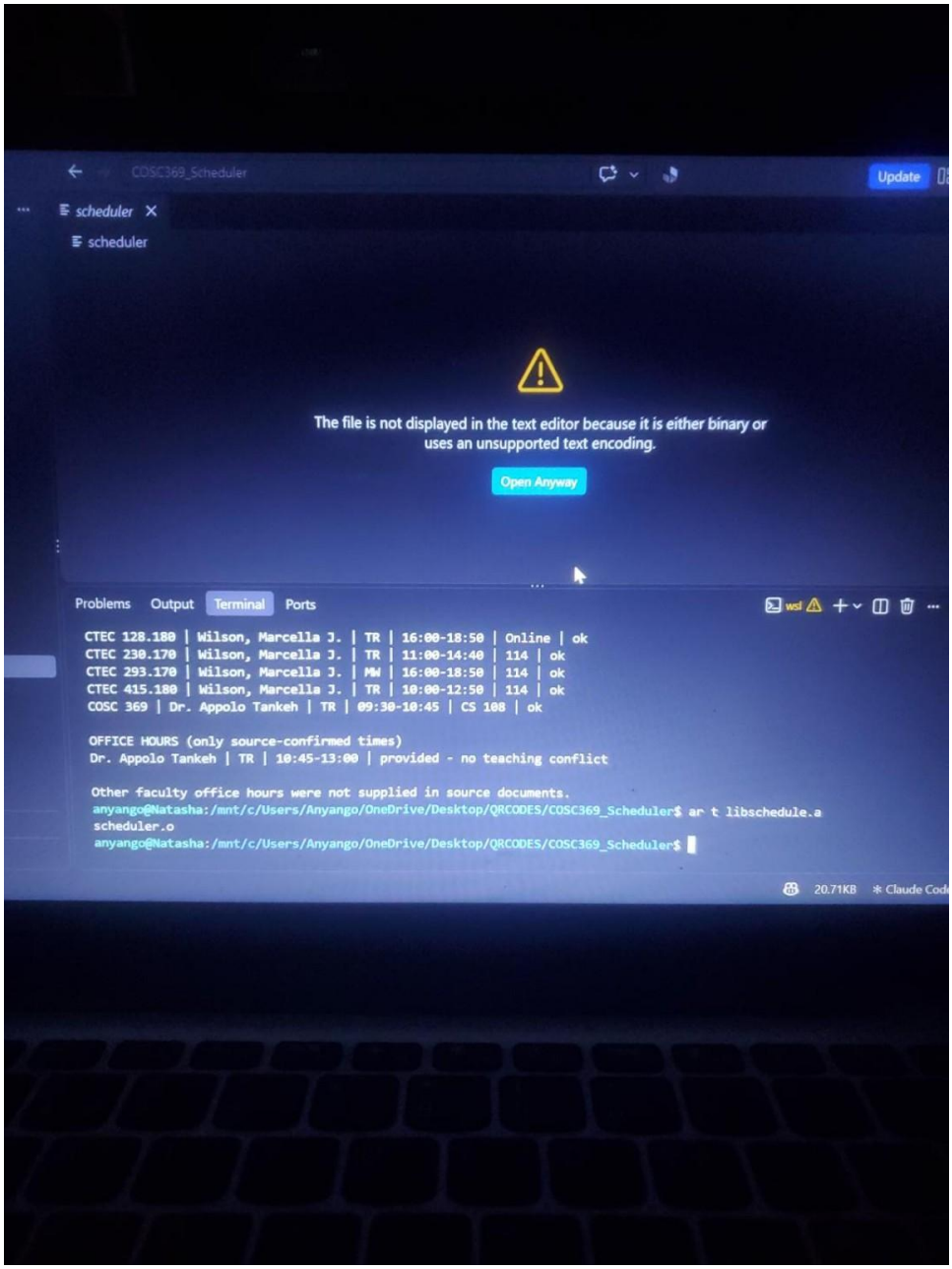


Figure 3. Static library inspection. The command ar t libschedule.a printed scheduler.o, demonstrating the object is in the archive.

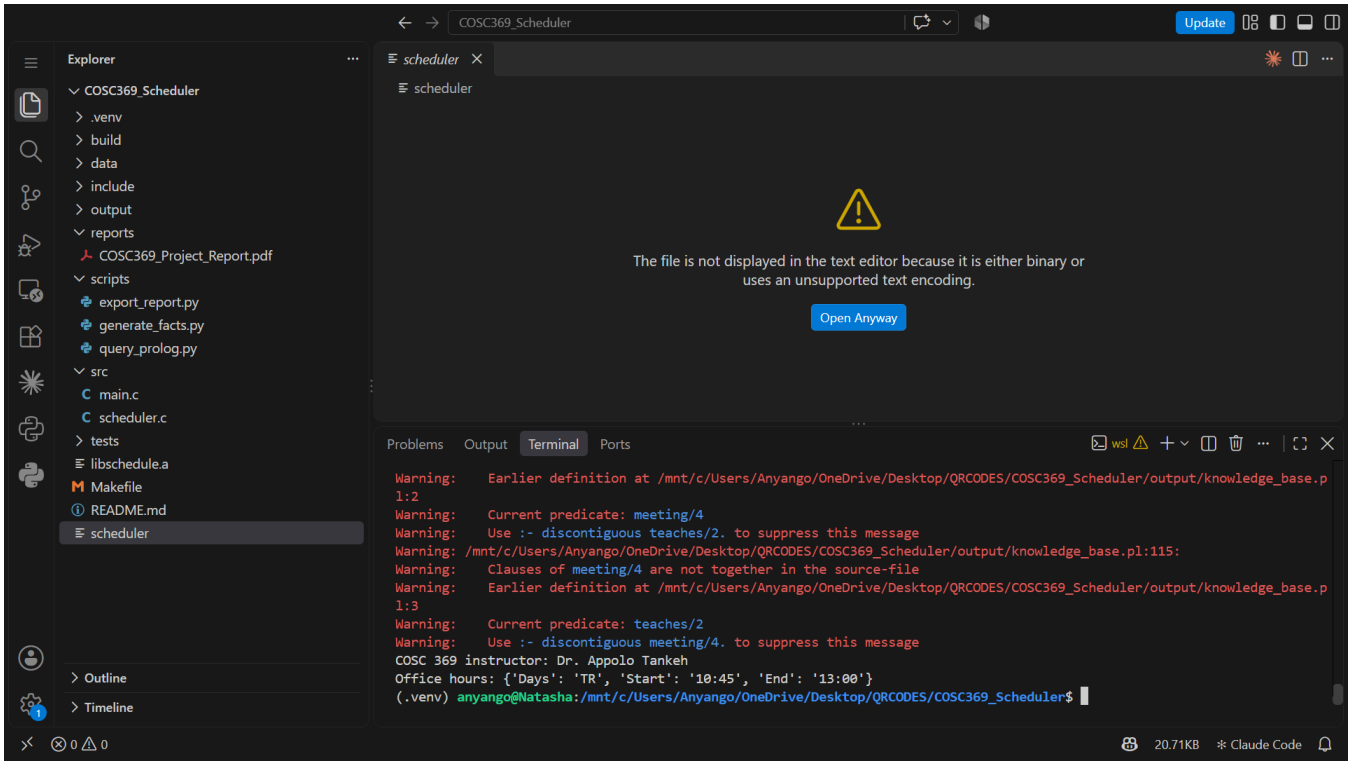


Figure 4. Generated teaching schedule and office-hours summary, including the COSC 369 row and the source-confirmed office-hour slot.

12. Interpretation, limitations and completion check

The reported 0 teaching overlaps means no conflicting teaching meetings were found by the implemented check for the tested data. It does not establish that all 29 faculty members have valid office hours. Only Dr. Tankeh has a source-confirmed slot; the remaining faculty availability is unspecified. The program flags five course-time entries for review without silently correcting them. The CTEC workbook labels the instructor column "Instructor 2025" despite a Fall 2026 file name, and the COSC 369 syllabus combines a Spring 2026 heading with Fall 2026 weekly dates. These source inconsistencies should be disclosed, not resolved by invention.

The source documents and student terminal evidence support a completed demonstration of the specified workflow: input processing, C build, static-library archive, sample schedule reporting, optional Prolog queries and tests. The lecturer did not expressly request screenshots, but they document actual execution. The input files do not provide a policy or availability windows for assigning new office hours to every professor; this report therefore does not claim universal automatic office-hour allocation.

Submission check: include the source ZIP, this report exported as PDF, README, two runtime CSV inputs, generated schedules, static library and the optional Python files. Review personal details, make sure the screenshots are legible, and follow the lecturer’s submission platform instructions. Course policy requires acknowledgment of any outside assistance.